

Course: Individual Software Development Process' 2026

Software Requirement Specification

By **COOKIN'BYDOING**

Chosen Irl Challenge: Project B: Lab Workflow & Request Management



Thanathas Puttapipat 6810545638

Anawin Chaichana 6810545964

Hugh Jaiden 6810546006

Thichanon Kangkachit 6810545611

Hugo Desvignes 6810546715

Table of Contents

1. Target Users.....	1
2. The System Goal.....	1
3. Goal Refinement via KAOS:.....	2
4. Use Case Diagram.....	3
5. Use Stories.....	4
6. User Requirement Specification.....	4
7. Activity Diagram.....	5
AD-1: Request submission and tracking.....	5
AD-2: Request Management	6
AD-3: Activity History	7
AD-4: Equipment Register.....	8
AD-5: Operational Dashboard	
8. System Requirement Specification.....	10
9. Software Architecture.....	11
Rationale.....	12
10. Data Storage Technology.....	13
Rationale.....	13
11. Development Environment.....	14
Rationale.....	14
12. Sequence Diagrams for All SRS.....	15
SQD-1: Lab Member Submits a Request.....	15
SQD-2: Lab Member Submits a Request(Validation Error Path).....	15
SQD-3: Lab Member Tracks Requests and Views Activity History.....	16
SQD-4: Staff Classifies and Progresses a Request.....	16
SQD-5: Assigned Staff Closes a Request.....	17
SQD-6: Staff Updates Equipment Availability.....	17
SQD-7: Lab Manager Opens the Operational Dashboard.....	18
13. Traceability Matrix.....	19

1. Target Users

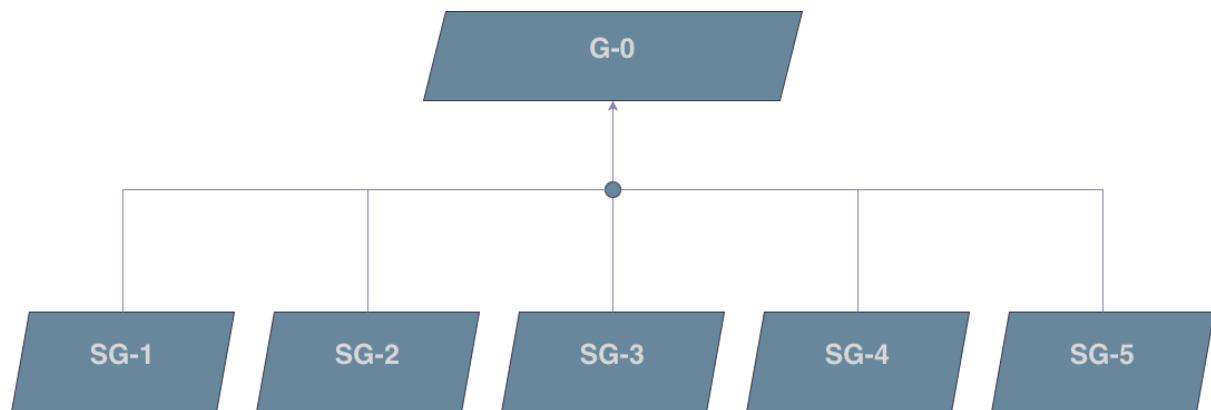
- **Labmember:** Submit requests for experiment setup, access permissions, and consumables. track the status of their own requests and see what equipment is available and what is currently down.
- **Teaching assistants:** can request to work on a submitted task; once accepted, they complete the work, update its progress, and stay in communication with the requester.
- **Lab manager:** Oversee the entire request queue, set priorities, allocate work across staff or accept or deny TAs request, maintain the equipment register, and review how lab resources are being used.
- **System administrator:** Manage user accounts and roles, maintain reference data such as request categories, and control access to administrative functions.

2. The System Goal

G-0: Maintain accurate and accessible tracking of lab requests and shared equipment.

The lab currently relies on fragmented tools such as email threads, chat messages, and verbal handoffs to manage experiment setup requests, access permissions, causing lab members to lose visibility into where their requests stand and causing staff to lose track of who owns what work and which equipment is actually available. For this iteration, the system scope is limited to request submission and tracking (lab-member-facing), request management (TA and lab-manager-facing), activity history and commenting (all parties), and equipment register and booking (staff- and lab-member-facing). User and role administration, reference-data configuration, and the operational dashboard with notifications are included as supporting features within this iteration. Billing, procurement, training/certification tracking, and integration with external calendar or inventory systems are explicitly out of scope for this iteration and will be addressed in later iterations.

3. Goal Refinement via KAOS:



- **Sub-Goal 1 (SG-1)**

Description: Achieve: Provide lab members with a single channel for submitting request

Rationale: Request currently arrive through email, chat so there's no reliable place for them

- **Sub-Goal 2 (SG-2)**

Description: Maintain: A clear and current status for every request.

Rationale: Each request needs a visible state (pending, in-progress, closed). Lab members can only see their request. TAs can see each request's priority and status, along with whether their own offer to take it on has been accepted or denied by the lab manager — so they know which requests they're assigned to and how urgent each one is.

- **Sub-Goal 3 (SG-3)**

Description: Achieve: Allow authorised staff to classify and progress requests.

Rationale: Only the lab manager should assign work, set priority, or change the state of a request. TAs have to send a request to the lab manager if they want to do that work.

- **Sub-Goal 4 (SG-4)**

Description: Avoid: Discussion and decisions being lost outside the request record.

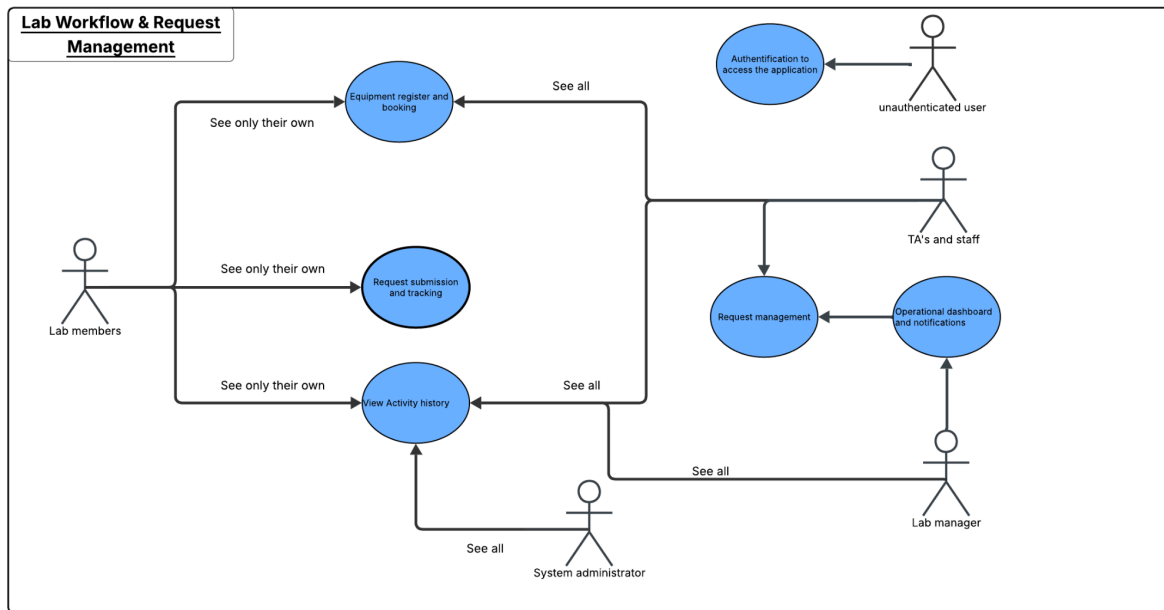
Rationale: Conversation scattered across email and chat makes follow-up untraceable and leaves new lab members without context.

- **Sub-Goal 5 (SG-5)**

Description: Maintain: An accurate record of laboratory equipment and its reservations.

Rationale: Members need to know what equipment exists, whether it is working, broken, or if it is already booked.

4. Use Case Diagram



Lab member— Submits requests, views and comments on their own only. Books available equipment.

Lab member (Guest) — Guest can have the maximum permissions of a Lab Member or less permissions depending on if it's an independent visitor or a company

TA / staff— Views the full queue, progresses assigned requests, and request to the lab manager to do the request

Lab manager — Views the operational dashboard and makes a decision (Accept/Denied) for TAs request to do the lab member's request.

System administrator — Manages accounts, roles, and reference data. Does not handle lab requests.

5. User Stories

User Story ID	Description	Acceptance Criteria
US-1	As a lab member, I want to be able to submit a request, register and edit equipment, and track them, so I can get what I need for the lab.	The lab member must be able to submit, register and edit equipment, and track their request.
US-2	As a TA, I want to see new requests submitted so I can request to work on it.	TAs should see a lab member's request and send a request to the lab manager if they want to handle that request.
US-3	As a lab manager, I want to be able to view and assign lab requests to TAs.	The lab manager would be able to view the overview and moderate the request.
US-4	As a system administrator, I want all parties to be able to report issues and submit requests involved with the system.	System administrators should fix issues when there's something wrong in the system.

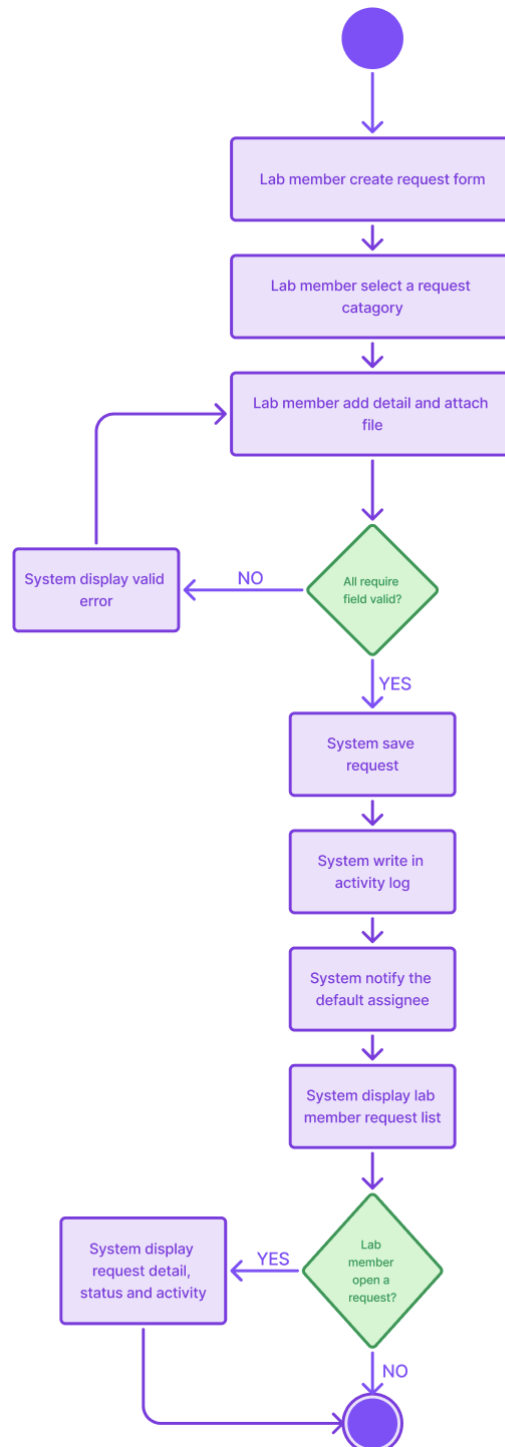
6. User Requirement Specification

ID	User Requirement
URS-1	Lab members shall be able to submit and track their request.
URS-2	TAs shall see lists of submitted requests, updated in real time and track their request.
URS-3	The system shall allow an authorised lab manager to see the overall request, assign requests to TAs and decide TAs's request.
URS-4	The system shall allow all users to report users or issues with the system.
URS-5	The system shall allow an administrator to view, handle, and close reported issues.

7. Activity Diagram

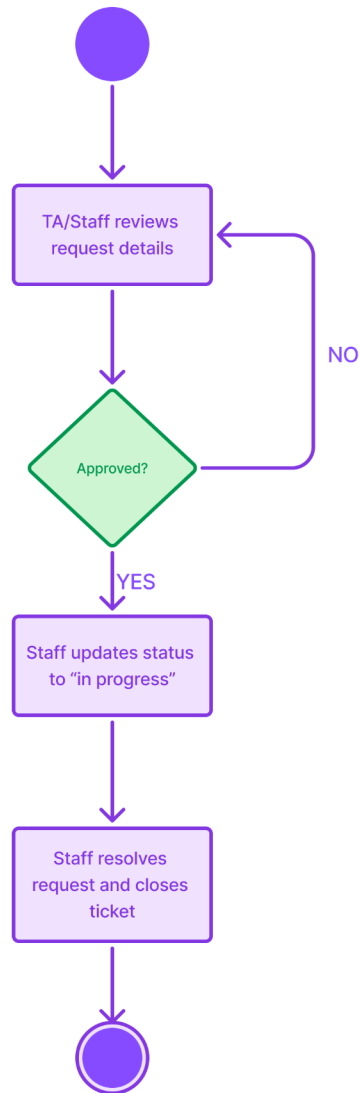
AD-1: Request submission and tracking

This diagram was used during elicitation to discover system responsibilities, decisions, and exception paths in the request submission process. Which were then converted into SRS-1, SRS-2, SRS-3 and SRS-4 below.



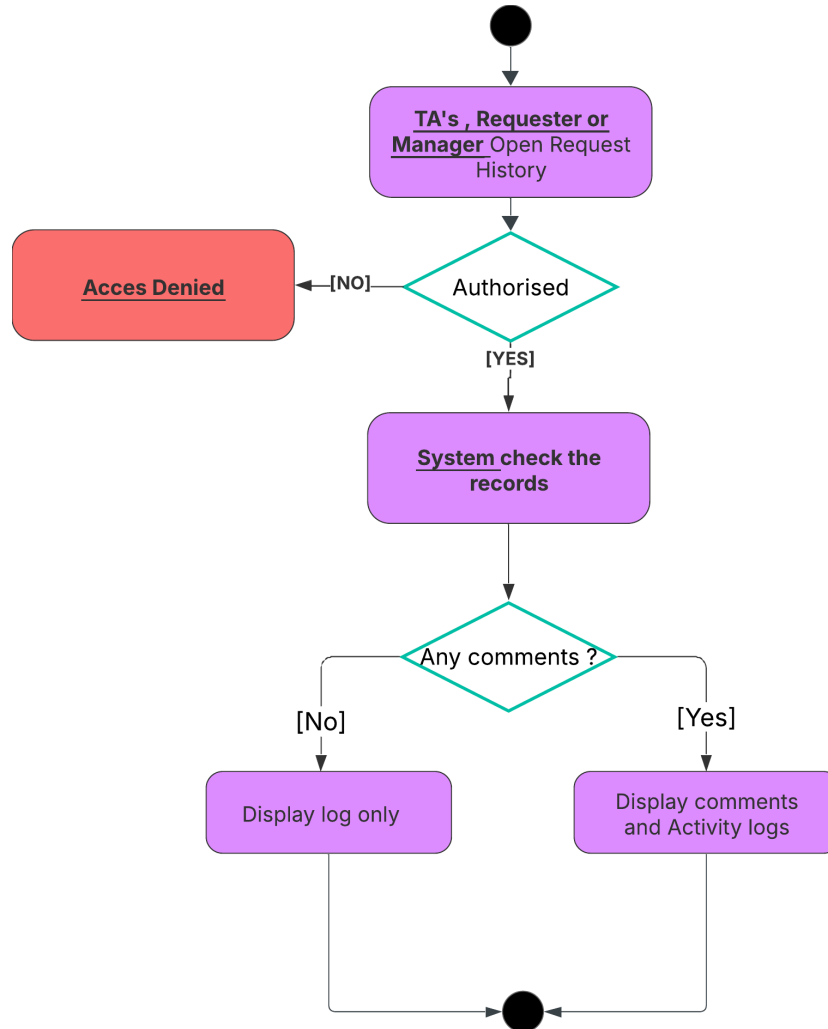
AD-2: Request Management

This diagram traces the staff handling flow. The TAs . Which were then converted into SRS-5, SRS-6, SRS-7 and SRS-8 below.



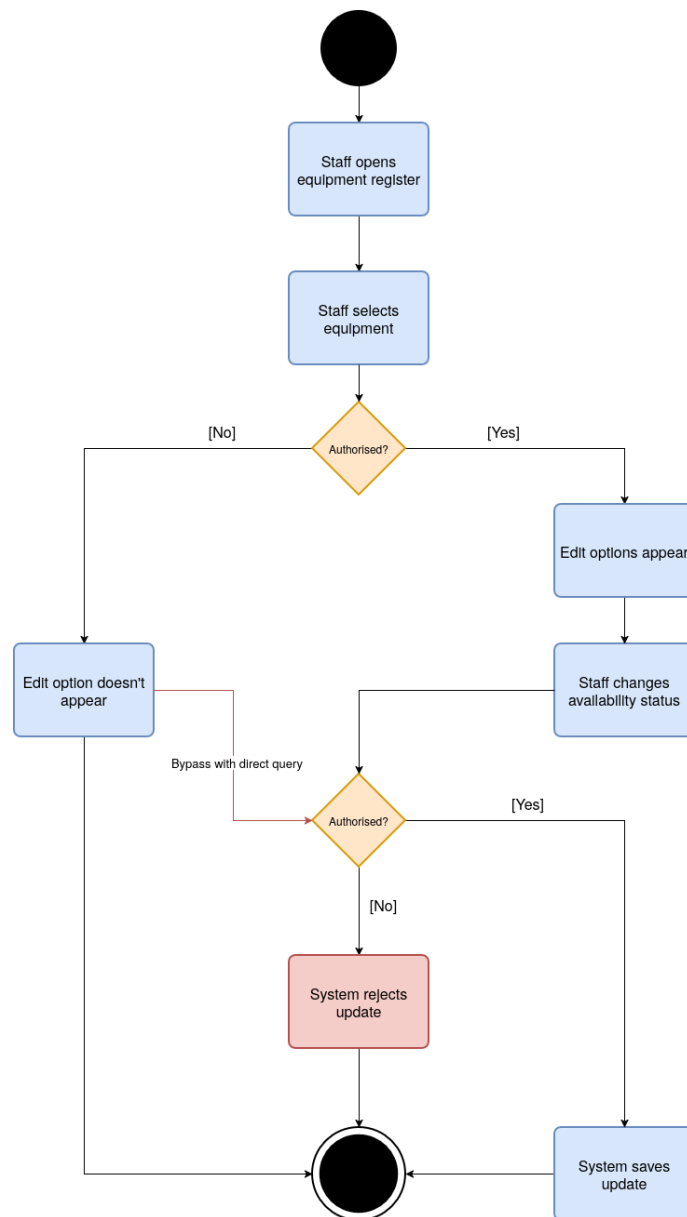
AD-3: Activity history

This diagram captures the authorisation check to open the Activity History , and which data is going to be displayed depending on the availability to derived SRS-9 and SRS-10.



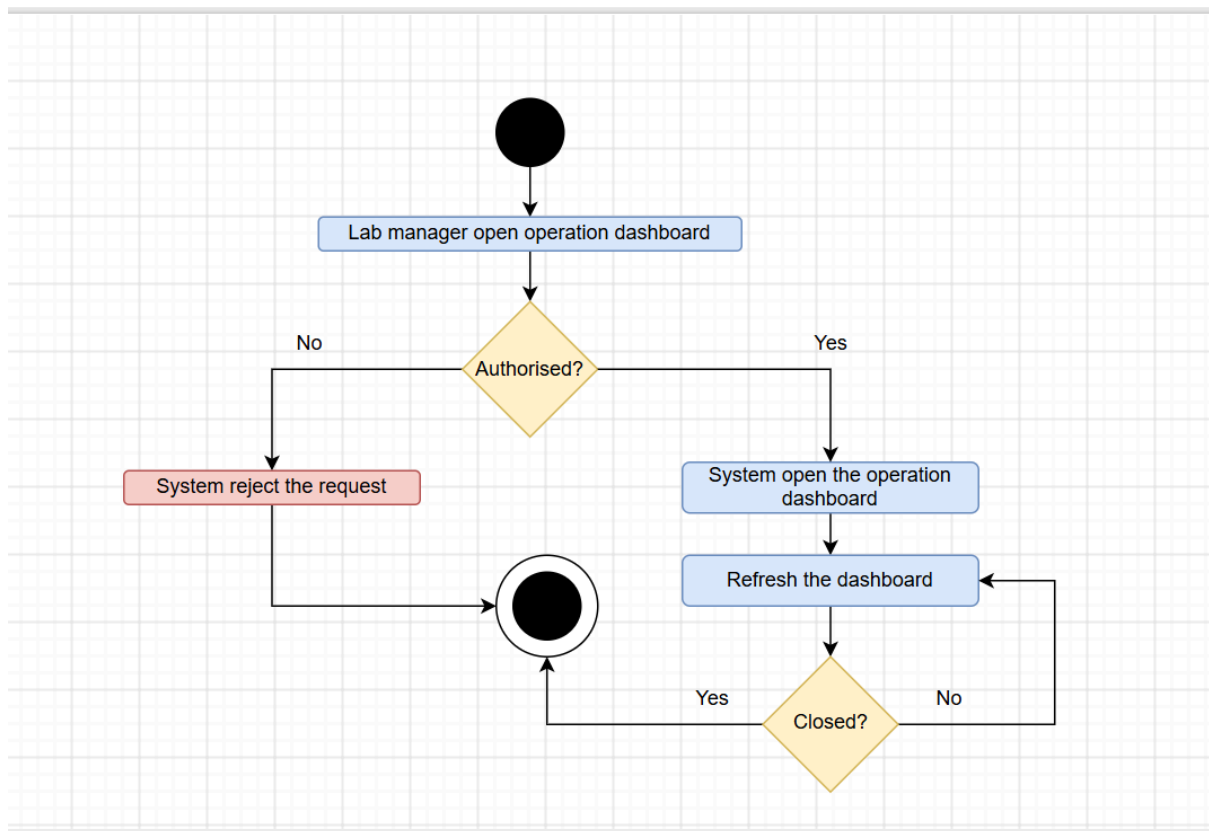
AD-4: Equipment Register

This diagram captures the equipment availability update flow and how the system rejects unauthorised users, used to derive SRS-11.



AD-5: Operational dashboard

This diagram captures the lab manager's authorisation check and the dashboard refresh loop, used to derive SRS-13 and SRS-14.



8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall allow a lab member to attach one or more files to a request.
SRS-2	The system shall display a validation error and shall not save the request when any required field is missing or invalid.
SRS-3	The system shall save a valid request with a unique request ID and an initial status of "Pending".
SRS-4	The system shall display to each lab member a list of their own requests, and shall show the detail, current status, and activity history of a request when it is opened.
SRS-5	The system shall allow a request to carry a type, description, and priority value.
SRS-6	The system shall allow the lab manager to change the status, priority, assignee of a request or decide the TAs's request.
SRS-7	The system shall allow assigned TAs to escalate a request to the lab manager, transferring ownership of the request.
SRS-8	The system shall allow the assigned TAs member to set a request status to "Closed". Then log the resolved status.
SRS-9	The system shall display the activity log for every request, and shall display the comment thread only when the request has at least one comment.
SRS-10	The system shall display the activity history of a request only to the lab member who submitted it, the TAs that handle that request, and the lab manager, and shall deny access to any other user.
SRS-11	The system shall display the equipment register, including each item's name, identifier, and availability status, to all authenticated users.
SRS-12	The system shall display equipment edit controls only to authorised TAs/Lab managers.
SRS-13	The system shall allow only an authorised lab manager to open the operational dashboard and shall reject the request otherwise.
SRS-14	The system shall display on the operational dashboard the current requests with their status, owner, and priority, and shall keep the displayed data refreshed until the lab manager closes the dashboard.

9. Software Architecture

Based on the SRS for lab request management system (Iteration 1 scope: request submission and tracking, request management , activity history and commenting, and equipment register and booking), architecture that seems fit is a Modular Architecture. This is a single deployable web application,organised into a clear separated layer, which is different from a flat monolith or a distributed microservices system.

Rationale

The system is being built and maintained by a small team of students and the current scope covers request submission and tracking, request management , activity history and commenting, and equipment register and booking. It looks a lot but it's actually many separated layers/systems working together. A modular architecture is overall the best architecture as monolithic is quite hard to manage when it's loosely organised and the Micro service scale is too high and probably causes unnecessary overhead. We just want a middle where the web app is a single unit, but has many layers so it's easier to manage and fit the small to medium scale of our project.

Component	Layer/Type	Description
Web / Mobile Client	Presentation (View)	Renders the request submission form, the lab member's own request list and request detail view, the staff request queue, the equipment register, the activity history and comment thread, and the lab manager's operational dashboard; displays validation errors, access-denied messages, and closed confirmations returned by the Controller (SRS-2, SRS-4, SRS-8, SRS-11, SRS-13).
Request Controller	Controller	Receives HTTP requests for creating a request, listing and opening requests, changing status/priority/assignee, escalating, and closing; invokes the Authorisation module before any write and delegates business rules to the Request Model (SRS-1 to SRS-8).
Equipment Controller	Controller	Handles requests to list the equipment register, retrieve edit controls, and submit availability updates or bookings; re-verifies authorisation on every submitted update, including updates that bypass the interface (SRS-11, SRS-12).
Dashboard Controller	Controller	Serves the operational dashboard, rejecting any user who is not an authorised lab manager, and serves the periodic refresh queries that keep the displayed queue current until the dashboard is closed (SRS-14).
Request Model / Business Logic	Model	Encapsulates request rules: validates required fields, assigns a unique request ID, sets the initial status "Submitted", holds type, description and priority, enforces the permitted status transitions (Submitted → In Progress → Closed), and transfers ownership on escalation (SRS-1 to SRS-8).
Activity & Comment Model	Model	Appends an immutable audit entry for every request creation and state change, stores comments against a request, and returns the activity log always and the comment thread only when at least one comment exists (SRS-5, SRS-9).
Equipment Model	Model	Maintains equipment records (name, identifier, availability status) and their reservations (SRS-11).
Authentication / Authorisation Module	Cross-cutting	Authenticates users and resolves their role (lab member, guest, TA/staff, lab manager, system administrator); evaluates every access decision server-side, including who may change a request (SRS-6), who may read a given request's activity history (SRS-10), who sees equipment edit controls (SRS-12), and who may open the dashboard (SRS-13).
Identity & Reference Data Module	Model / Administration	Allows the system administrator to manage user accounts and role assignments and to maintain reference data such as request categories and equipment types, without granting access to lab request handling.
Lab Database	Persistence	Stores users and roles, requests, assignments, comments, activity log entries, equipment records, and reservations, and serves the queries behind the member request list, the staff queue, and the dashboard.
Attachment Store	Persistence	Stores the one or more files attached to a request, referenced from the request record by metadata rows (SRS-1).

10. Data Storage Technology

For this project we chose to use RDBMS (Relational Database Management System) , the most famous one : MySQL

Rationale

Component	Data Storage Technology	Description
Lab Database	RDBMS : MySQL	Normalised schema for user, role, request, request_assignment, comment, attachement, activity_log, equipment and reservation, with foreign keys, enumerated status/priority columns, and NOT NULL / CHECK constraints supporting SRS-1 to SRS-15.
Activity Log Table	RDBMS (append-only table)	Immutable audit entries written in the same transaction as the request change they describe, so the history displayed under SRS-9 is always complete and ordered.
Attachment Store	File location stored in the Attachment table + Object Storage	Holds the files attached to requests; the database stores filename, size, content type, uploader and storage key, and the request record links to them (SRS-1).

Only one data store is required for the current SRS scope, since both customer-facing menu viewing and administrator menu management operate on the same menu-item entity. Should future iterations introduce less-structured data (e.g., free-form announcements or activity logs), a supplementary NoSQL document store could be added at that time without disrupting the relational menu store.

11. Development Environment

The recommended development and deployment environment is a containerised environment using Docker, rather than bare metal.

Rationale

The project is developed by five people working on their own laptops across different operating systems, and then demonstrated on a different machine again. The system depends on a specific RDBMS version, an application runtime, and an attachment volume that differs subtly between versions and between a local install and the deployment target. Packaging the application, the database, and their configuration as Compose services means every team member runs an identical stack from one command, so a validation or authorisation bug can be reproduced on any machine instead of only on the laptop that found it.

For deployment, a single Compose file plus a volume for attachments and database data can be deployed to a lab VM or a hosted server without reinstalling dependencies or reconfiguring the host, and can be reset or rolled back quickly. Bare-metal deployment would tie the system to one machine's configuration, and make it fragile to reproduce if the lab's hardware changes or a second device is added.

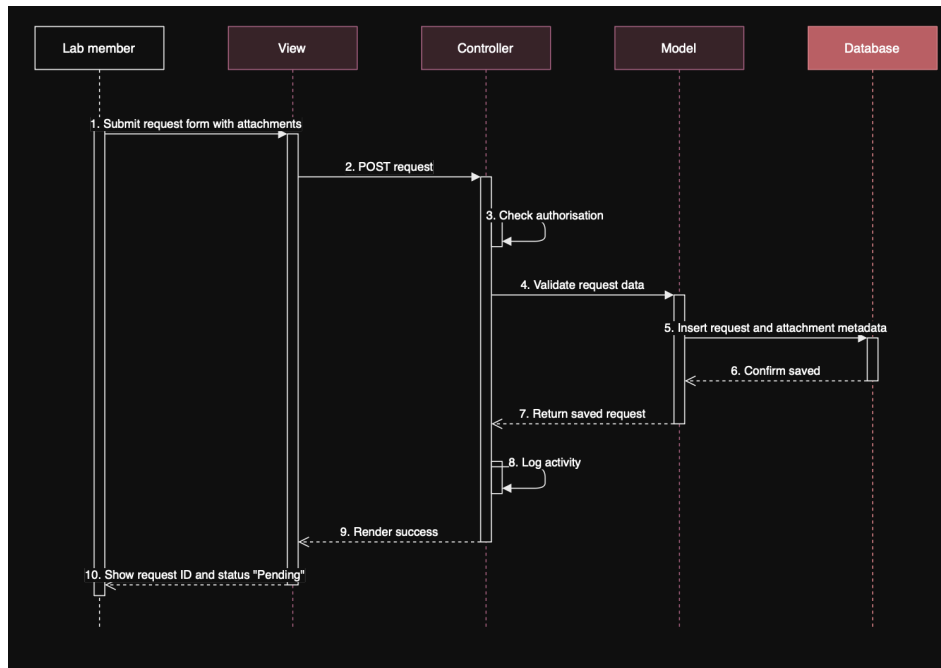
Component	Development Environment	Description
Application Server (Controller + Models)	Docker containers (via Docker compose)	Runs the modular MVC application in a reproducible image, identical between each developer's laptop, in CI, and on the lab's server.
Lab Database (RDBMS)	Docker container (via Docker Compose)	Runs MySQL as its own service with a named volume for data, pinned to one version so schema constraints and transaction behaviour are identical everywhere.
Local Development Machine	Docker Desktop/Virtual machine	Used by each of the five developers to build, run and test the full stack locally against the same service definitions as production.
Deployment Host	Docker Engine	Hosts the Compose stack for the demonstration and for lab use, reproducible from the repository.

12. Sequence Diagrams for All SRS

The following sequence diagrams are derived from the modular MVC architecture in Section 9 and together cover SRS-1 to SRS-15 and all five use cases

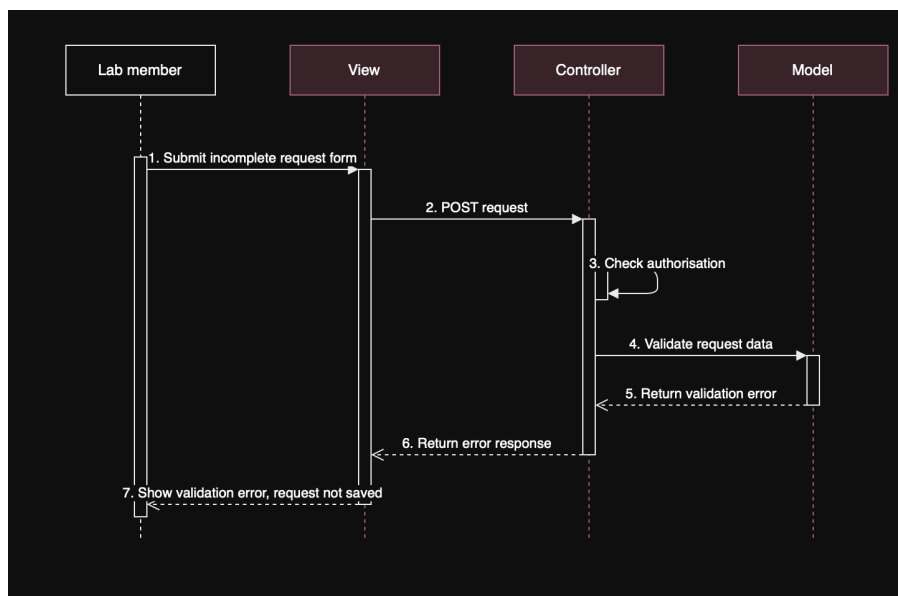
SQD-1: Lab Member Submits a Request

Traced requirements: SRS-1, SRS-3, SRS-5



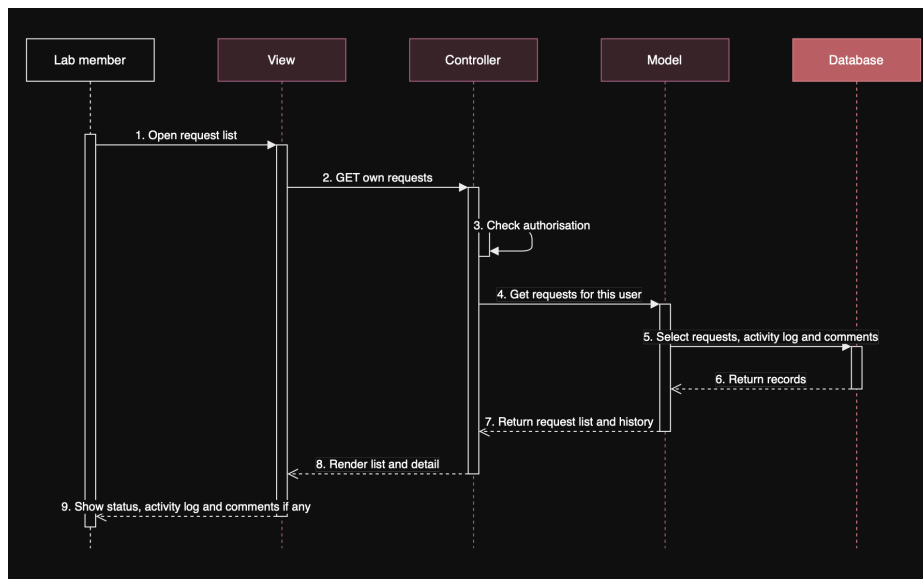
SQD-2: Lab Member Submits a Request (Validation Error Path)

Traced requirements: SRS-2



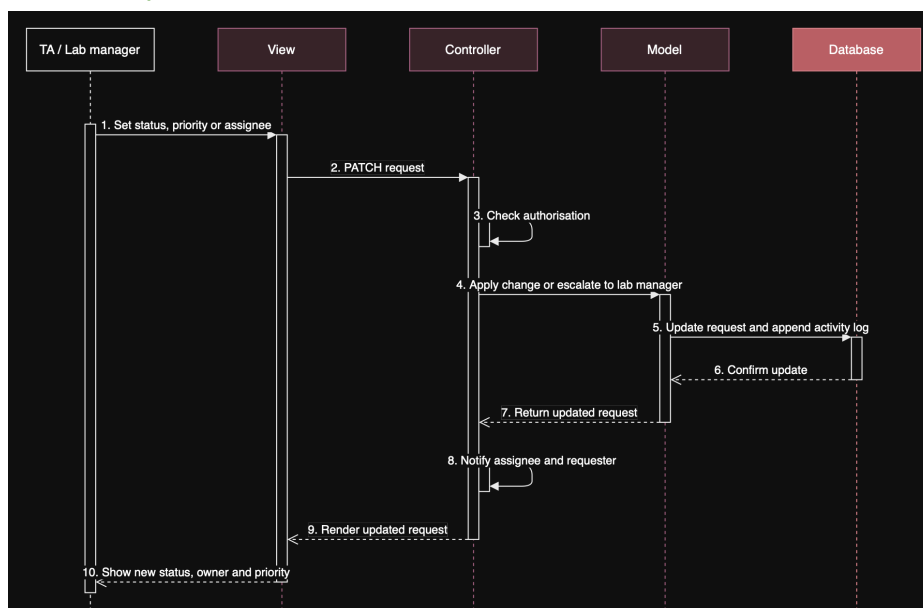
SQD-3: Lab Member Tracks Requests and Views Activity History

Traced requirements: SRS-4, SRS-9, SRS-10



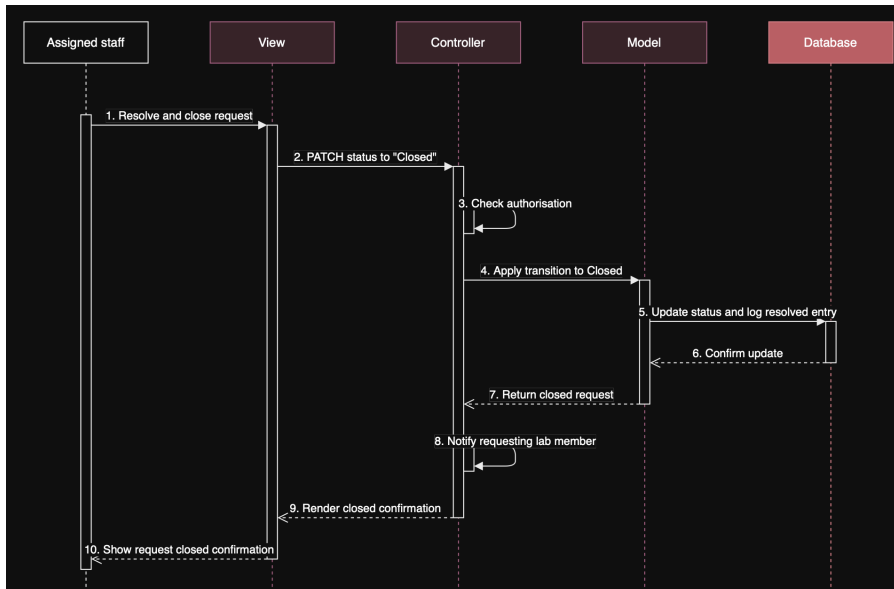
SQD-4: Staff Classifies and Progresses a Request

Traced requirements: SRS-5, SRS-6, SRS-7



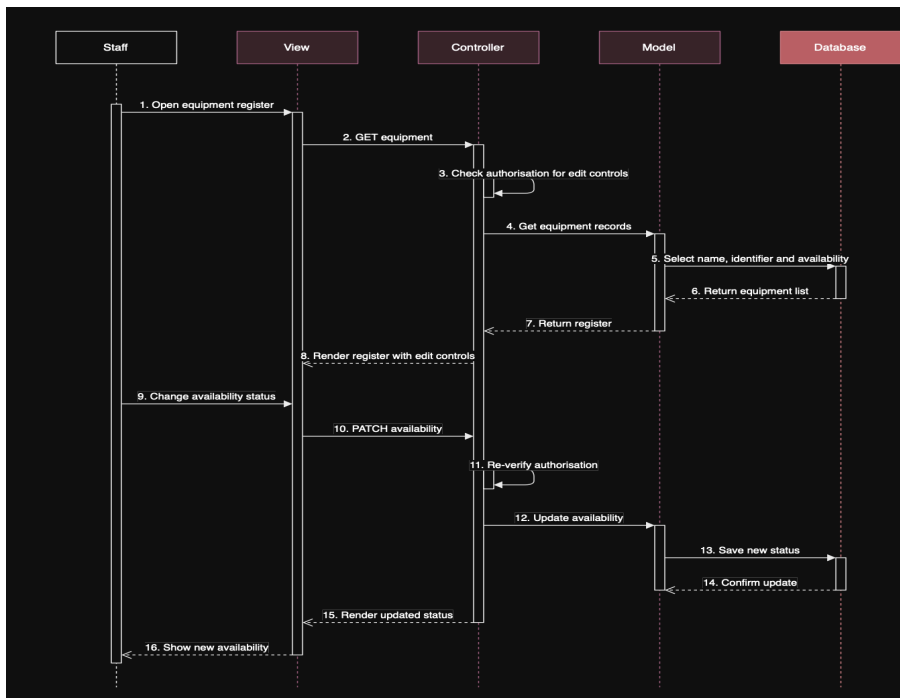
SQD-5: Assigned Staff Closes a Request

Traced requirements: SRS-8



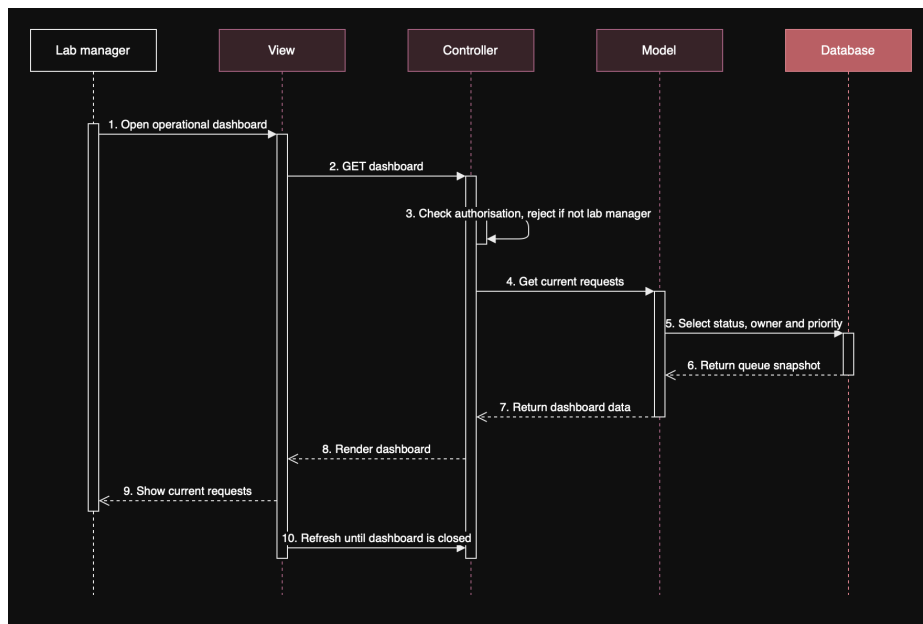
SQD-6: Staff Updates Equipment Availability

Traced requirements: SRS-11, SRS-12



SQD-7: Lab Manager Opens the Operational Dashboard

Traced requirements: SRS-13, SRS-14



13. Traceability Matrix

From Sub-Goal to SRS

From SRS to Sequence Diagram

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	UC-1 Request Submission	US-1	URS-1	AD-1	SRS-1, SRS-2, SRS-3, SRS-4
SG-2	UC-2 Request Management	US-3, US-2	URS-1, URS-2	AD-2	SRS-5, SRS-6, SRS-7, SRS-8
SG-3	UC-3 Activity History	US-1	URS-3	AD-3	SRS-9, SRS-10
SG-4	UC-4 Equipment Register	US-1	URS-4	AD-4	SRS-11, SRS-12
SG-5	UC-5 Operational Dashboard	US-4	URS-5	AD-5	SRS-13, SRS-14

SRS	SQD
SRS-1	SQD-1
SRS-2	SQD-2
SRS-3	SQD-1
SRS-4	SQD-3
SRS-5	SQD-1, SQD-4
SRS-6	SQD-4
SRS-7	SQD-4
SRS-8	SQD-5
SRS-9	SQD-3
SRS-10	SQD-3
SRS-11	SQD-6
SRS-12	SQD-6
SRS-13	SQD-7
SRS-14	SQD-7